

Minimapas de Código-Fonte como Abordagem Visual para Análise de Software Baseada em IA

Exame de Qualificação de Doutorado

Munif Gebara Junior

Orientador: Prof. Dr. Yandre Maldonado e Gomes da Costa

Coorientadores: Prof. Dr. Igor Scaliante Wiese · Prof. Dr. André F. R. Cordeiro

Programa de Pós-Graduação em Ciência da Computação · Universidade Estadual de Maringá

Maringá, 3 de setembro de 2026

Roteiro

- O problema: escala, heterogeneidade e confidencialidade
- A proposta: minimapas como assinaturas visuais anonimizadas
- Lacuna na literatura e posicionamento
- Hipótese, questões de pesquisa e objetivos
- Metodologia
- Três estudos concluídos e a ferramenta MinimapSignatures
- Limitações, plano de pesquisa e cronograma

Software moderno é grande, heterogêneo e difícil de analisar

- Um único produto combina várias linguagens, frameworks, arquivos gerados, configuração, scripts, testes e infraestrutura
- Compreensão de programas é atividade central da manutenção: localizar artefatos, reconhecer responsabilidades, avaliar conformidade com as convenções do projeto
- Análise estática e visualização ajudam, mas costumam exigir parsers, gramáticas, regras e configuração especializada
- Representações de código para aprendizado de máquina são poderosas, porém dependem de tokenizadores, árvores sintáticas ou grandes corpora de pré-treinamento

O código-fonte carrega informação estratégica de negócio

- Identificadores, constantes numéricas, fórmulas e comentários codificam regras de negócio
- A maioria das representações para aprendizado de máquina retém exatamente essa camada lexical
- Enviar o arquivo a um serviço externo de análise expõe propriedade intelectual — motivo suficiente para uma organização descartar a ferramenta

profitMargin = grossRevenue * 0.2345

Quem lê o identificador e a constante juntos infere o cálculo interno da empresa.

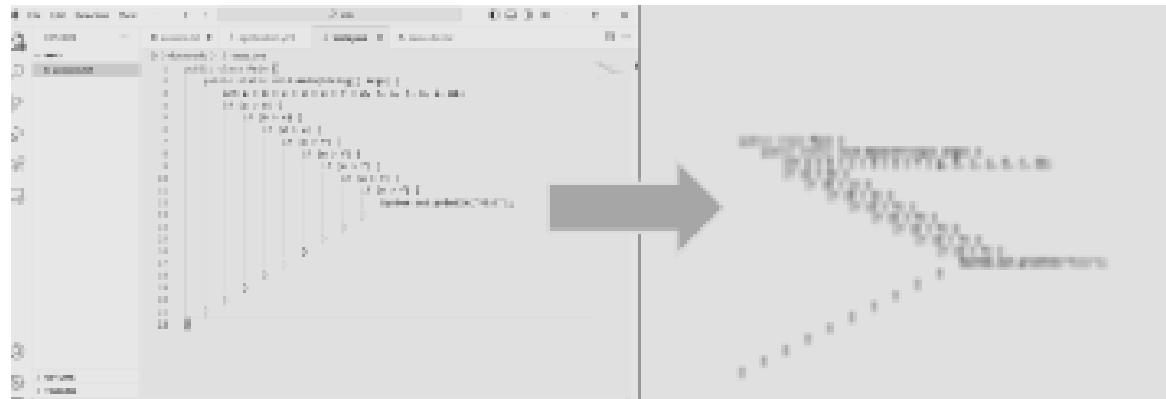
O problema

Projetos de software contêm sinais estruturais e estilísticos úteis para classificação, compreensão, avaliação de qualidade e apoio ferramental — mas as representações existentes ou exigem processamento específico de linguagem, ou retêm o conteúdo lexical que expõe lógica de negócio, constantes proprietárias e conhecimento estratégico de domínio.

A pergunta que segue: uma representação visual do arquivo pode servir de abstração intermediária — simples de gerar em muitas linguagens, compacta, discriminativa e capaz de suprimir o léxico?

Minimapas de código-fonte

- Cada caractere vira um pixel; cada linha, uma fileira da imagem
- A imagem preserva indentação, comprimento de linha, densidade de pontuação e organização em blocos
- Inspirado no minimapa que os editores já exibem — mas produzido para ser lido por um modelo



Do arquivo-fonte ao minimapa: a mesma geometria, sem o texto

Duas mudanças de representação

1. De uma para duas dimensões

O código é armazenado e analisado como sequência linear de caracteres ou tokens.

O desenvolvedor, porém, escreve e lê uma página bidimensional: a indentação forma contornos, os blocos têm forma, as linhas em branco criam ritmo.

O minimapa devolve essa segunda dimensão, que o modelo de sequência trata como espaço em branco incidental.

2. Do humano para a máquina

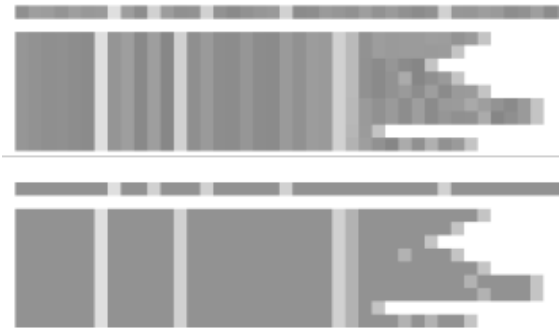
A visualização de software é historicamente projetada para inspeção humana.

Aqui a imagem é insumo de modelo: compacta, reproduzível e discriminativa, não expressiva e interativa.

O humano volta ao circuito no nível das predições e das explicações, não da figura.

A anonimização é parte da representação

Classe de caractere	Faixa ASCII	Pixel	Efeito
Controle e espaços	< 32	0	Preserva indentação e linhas em branco
Dígitos	48–57	53	Suprime literais numéricos
Maiúsculas	65–90	77	Suprime identificadores
Minúsculas	97–122	109	Suprime identificadores
Demais imprimíveis	restante	ASCII	Preserva operadores e delimitadores



Antes e depois

Identificadores, literais e comentários desaparecem. Indentação, pontuação, operadores e organização em blocos permanecem.

Mapeamento da literatura: 56 estudos selecionados

- Cinco bibliotecas digitais: IEEE Xplore, ACM DL, Scopus, SpringerLink e Google Scholar
- Triagem em estágios: 270 candidatos → 155 → 96 → 56 estudos; dois revisores, triagem cega no Rayyan
- Famílias: visualização de software · representações para aprendizado de máquina · código como dado visual · qualidade e code smells · explicabilidade · apoio ferramental

A lacuna central

O mapeamento não identificou representação de código projetada para suprimir conteúdo lexical proprietário e, ainda assim, permanecer útil como entrada de aprendizado de máquina. Ofuscação protege o binário entregue; varredura de segredos remove credenciais; aprendizado federado protege o dado no protocolo — nenhum atua na representação.

Hipótese

Minimapas anonimizados de código-fonte, entendidos como projeções bidimensionais destinadas ao consumo por máquina, preservam informação estrutural e visual suficiente para sustentar análise de software escalável e independente de linguagem, reduzindo a dependência de parsers e de regras definidas por especialistas.

Três fronteiras declaradas

- Não substitui análise semântica: significado depende de construções, dependências e domínio
- Não é garantia formal de privacidade: é supressão da camada lexical que carrega propriedade intelectual
- Não é representação única para toda a engenharia de software: é a primeira camada de uma família de tarefas

Questões de pesquisa

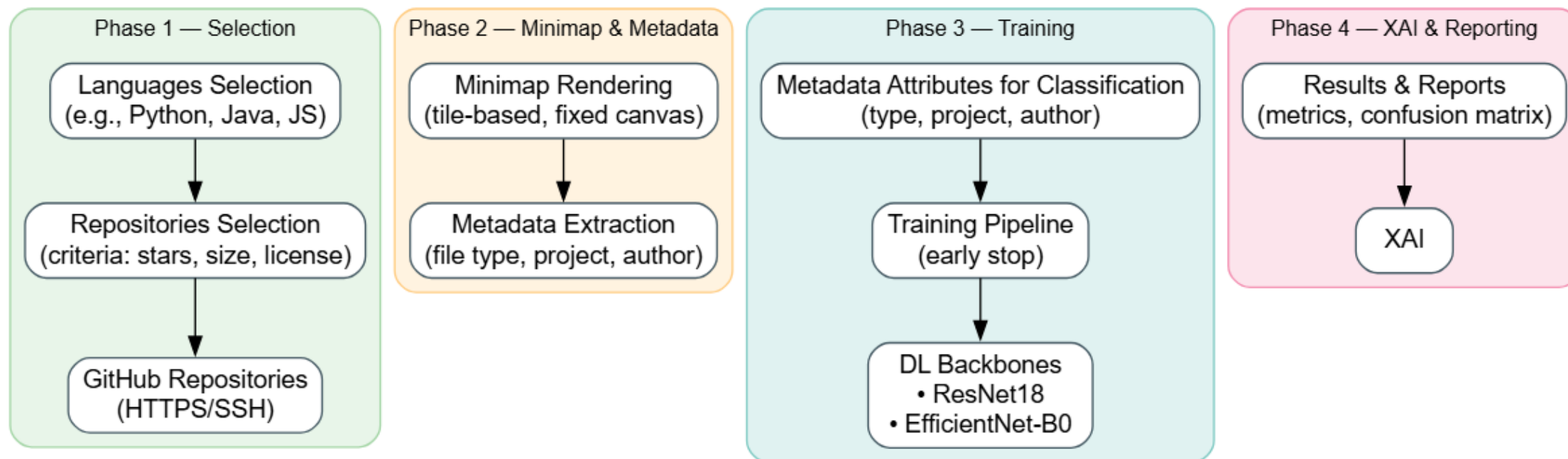
- RQ1** Em que medida minimapas preservam pistas estruturais discriminativas para classificar artefatos entre linguagens?
- RQ2** Como descritores de textura artesanais se comparam a representações visuais aprendidas?
- RQ3** Quais regiões visuais os modelos usam, e como a IA explicável apoia a interpretação dessas decisões?
- RQ4** Como a anonimização afeta o equilíbrio entre confidencialidade e desempenho preditivo?
- RQ5** Como integrar a análise por minimapas ao fluxo de trabalho real em IDEs?

Objetivos

Objetivo geral: definir, avaliar e operacionalizar uma abordagem de minimapas de código-fonte para análise visual de software.

- Definir método reprodutível de geração de minimapas a partir de repositórios heterogêneos
- Projetar e avaliar estratégias de anonimização que suprimam o léxico preservando pistas estruturais
- Construir datasets e catálogos que sustentem aprendizado supervisionado sobre minimapas
- Comparar descritores de textura artesanais e modelos visuais profundos
- Investigar IA explicável para identificar as regiões que sustentam as predições
- Desenvolver apoio ferramental integrado ao ambiente de desenvolvimento
- Estender a avaliação rumo a qualidade, code smells e desvio arquitetural

Metodologia: um pipeline modular



Seleção e descoberta de arquivos → anonimização → renderização → catálogo → treinamento e avaliação → interpretação e integração à IDE

Cada etapa roda isolada: a geração não exige treinamento, o mesmo catálogo serve a vários objetivos e o modelo treinado vale tanto no experimento quanto na inferência dentro da IDE.

Catálogo e controle de vazamento

O catálogo

- Identificador do artefato e do repositório
- Caminho relativo hasheado
- Extensão ou tipo inferido e linguagem
- Rótulo de autor ou committer
- Partição de treino, validação ou teste
- Modo de anonimização aplicado

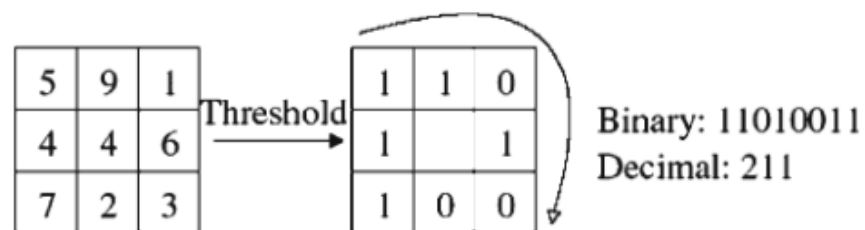
Três estratégias de partição

- Estratificada por arquivo — útil no desenvolvimento inicial, vulnerável a vazamento entre repositórios
- Ciente de repositório — necessária para afirmar generalização entre projetos
- Temporal — simula prever o futuro a partir do passado, exige metadado histórico confiável

Controle: hash de conteúdo, remoção de duplicatas e marcação de arquivos gerados.

Estudo 1: descritores de textura

- Corpus: dois sistemas proprietários sob acordo de confidencialidade — um ERP industrial e um sistema do governo federal
- 42.820 minimapas em 28 classes; recorte controlado com 7 classes e 1.000 amostras cada
- Descritor: Local Binary Patterns (8 vizinhos, raio 2, uniforme não rotacional) → histograma de 59 bins
- Classificadores: K-vizinhos, Random Forest e SVM; variantes de tamanho fixo (128×128) e variável

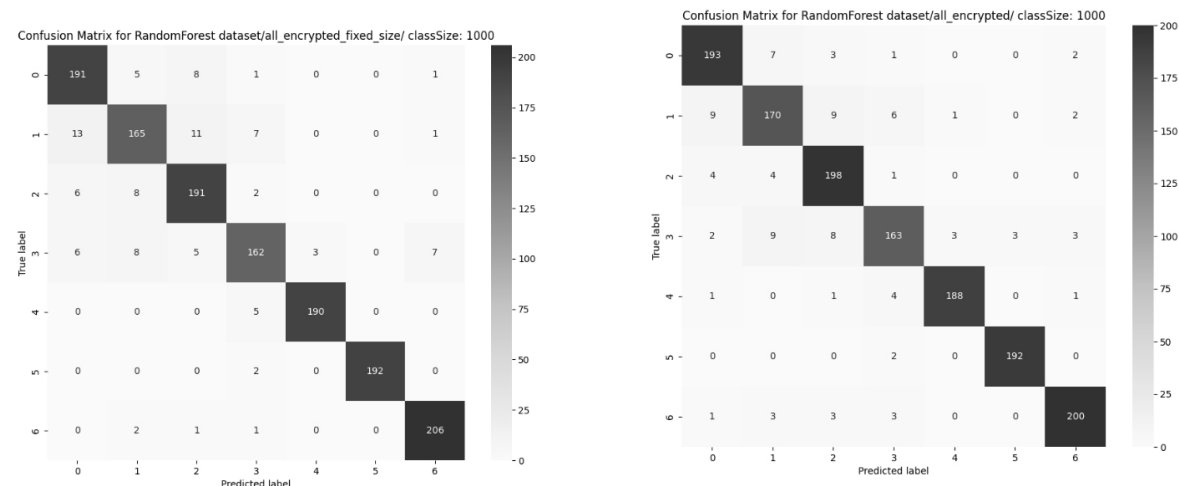


O operador LBP: cada pixel vira um código binário da sua vizinhança

Estudo 1: minimapas já separam classes sem parser

Classificador	Tamanho variável	Tamanho fixo
K-vizinhos	0,79	0,78
Random Forest	0,93	0,93
SVM	0,89	0,88

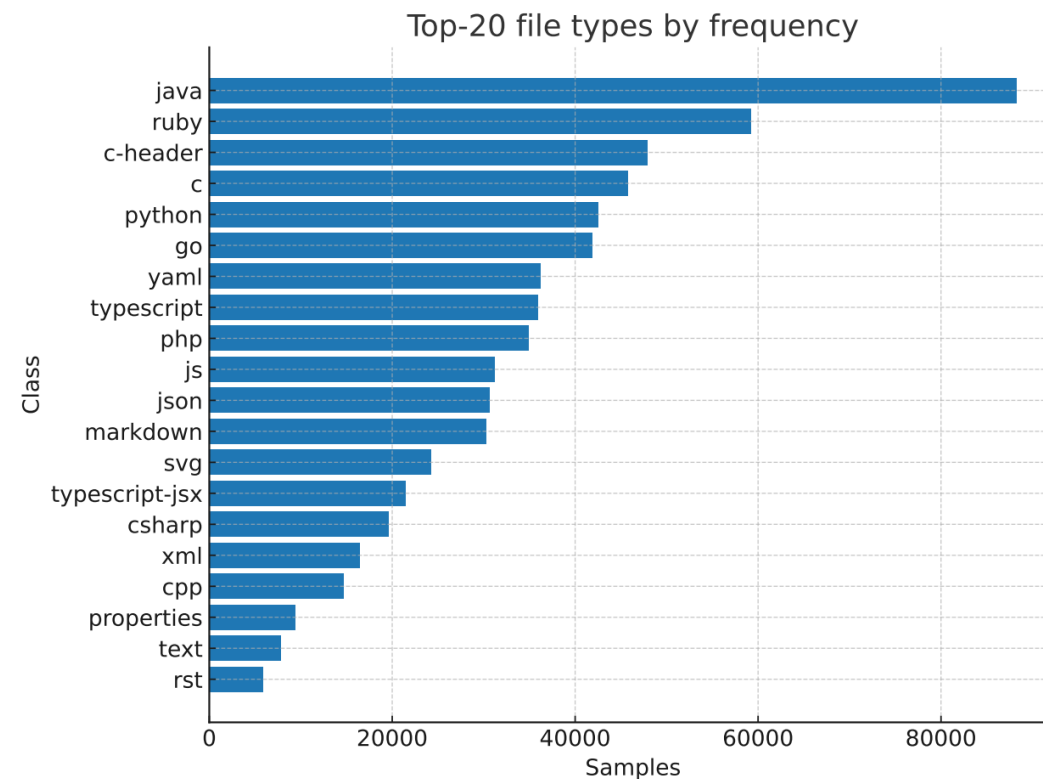
- Acurácia no conjunto de teste; a métrica única é a que o artigo reporta
- Random Forest chega a ~93% sem qualquer análise de linguagem
- O tamanho fixo custa no máximo um ponto — viabiliza as redes convolucionais



Matrizes de confusão do Random Forest: JSON e SVG separam bem; classes Java se confundem

Estudo 2: a abordagem em escala

- 685.906 minimapas de 100 projetos de código aberto em 10 linguagens
- 63 classes de Tipo, 99 de Projeto e 97 de Autor; imagens 128×128 em escala de cinza
- ResNet-18 e EfficientNet-B0, do zero e com pré-treinamento ImageNet
- Sem aumento de dados; parada antecipada; sementes fixas



Distribuição das classes de Tipo mais frequentes

Estudo 2: resultados e o que eles não dizem

Objetivo	Acurácia	ROC-AUC macro	ROC-AUC micro
Tipo	0,9333	$\geq 0,99$	$\geq 0,99$
Projeto	0,8604	$\geq 0,99$	$\geq 0,99$
Autor	0,8705	$\geq 0,99$	$\geq 0,99$

EfficientNet-B0 com pré-treinamento ImageNet, partição de validação

Três ressalvas que faço antes que a banca faça

- Com ~ 100 classes avaliadas um-contra-resto, o ROC-AUC é dominado por negativos fáceis: AUC alto indica separabilidade, não decisão confiável
- A partição não é disjunta por repositório — para Tipo e Autor, o resultado é validação intra-corpus
- Macro-F1, recall por classe e resultados de teste ainda precisam ser consolidados; o pipeline já os produz

O que os modelos olham



Integrated Gradients, 32 passos, baseline cinza médio, atribuição para a classe predita

- Tipo: bandas estruturais, regiões de indentação e padrões de delimitadores
- Projeto: convenções do repositório, cabeçalhos, trechos gerados e boilerplate
- Autor: atribuição difusa — o estilo é distribuído e difícil de localizar

Serve também de alerta: se Projeto depende de boilerplate, pode não generalizar para repositórios com outros modelos de arquivo.

A anonimização custa desempenho?

- Toda a afirmação de confidencialidade da tese depende desta resposta
- Até recentemente ela vinha de uma única comparação informal, não documentada — insuficiente para sustentar o argumento
- O método oferece uma comparação incomumente limpa: a anonimização é um mapeamento determinístico por caractere, aplicado na renderização
- O mesmo arquivo é renderizado duas vezes: mesmas dimensões, mesmo layout, mesmo rótulo — muda apenas o alfabeto de intensidades
- As duas versões saem na mesma passada, o que garante pareamento instância a instância

Desenho do experimento

Configuração

- Corpus: sistema do governo federal, versão mais recente
- 12 classes por convenção de nomenclatura; 10.923 pares de minimapas
- Descritor LBP de 59 bins e Random Forest
- Validação cruzada estratificada 10×5 , com os mesmos splits para as duas representações

Duas perguntas, dois testes

- Existe diferença detectável? Teste t pareado com correção de Nadeau–Bengio e teste de McNemar
- A diferença é pequena o bastante para ser irrelevante? TOST contra margem declarada antes do experimento
- Margem de equivalência: dois pontos percentuais de acurácia

Ausência de diferença significativa não estabelece equivalência: pode ser apenas falta de poder.

A anonimização não custou acurácia

Medida	Valor
Acurácia, renderização original	0,8015 ± 0,0069
Acurácia, renderização anonimizada	0,8076 ± 0,0063
Diferença (original – anonimizada)	–0,0060
Intervalo de confiança de 95%	[–0,0123 ; +0,0003]
Teste de diferença (t pareado corrigido)	p = 0,060
Equivalência em ±2 p.p. (TOST)	p = 2,4 × 10 ^{–5}

Leitura

- O ponto estimado favorece a versão anonimizada em 0,6 p.p.
- O limite superior do intervalo coloca a maior perda compatível com os dados em praticamente zero
- McNemar detecta diferença em 6 das 10 repetições, sempre na mesma direção: 5.239 acertos exclusivos da versão anonimizada contra 4.580 da original

E o que exatamente a transformação remove?

- 12,3 milhões de caracteres do mesmo corpus, medidos sem treinar nada
- Alfabeto: 152 → 37 intensidades distintas
- Entropia por pixel: 5,444 → 2,358 bits
- Redução de 56,7% da informação por pixel
- Apenas 25,9% dos caracteres passam intactos — exatamente pontuação, delimitadores e operadores
- Varia por tipo: 62,6% em Java, 48,0% em SVG

-56,7%

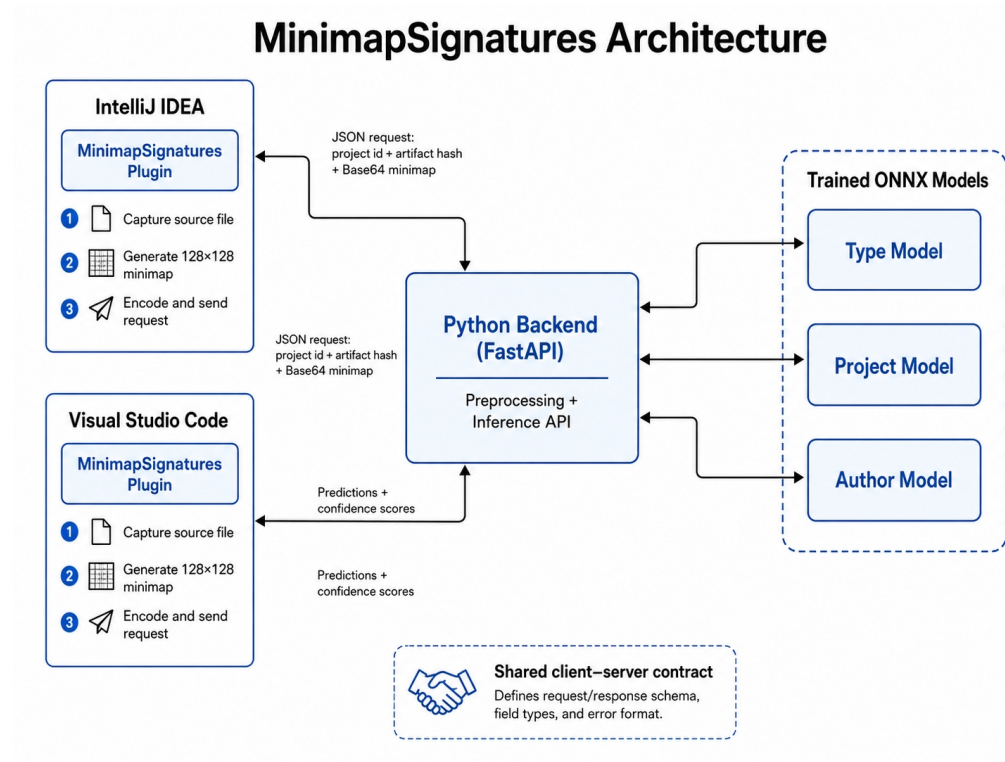
da entropia por pixel é descartada

e a acurácia não cai.

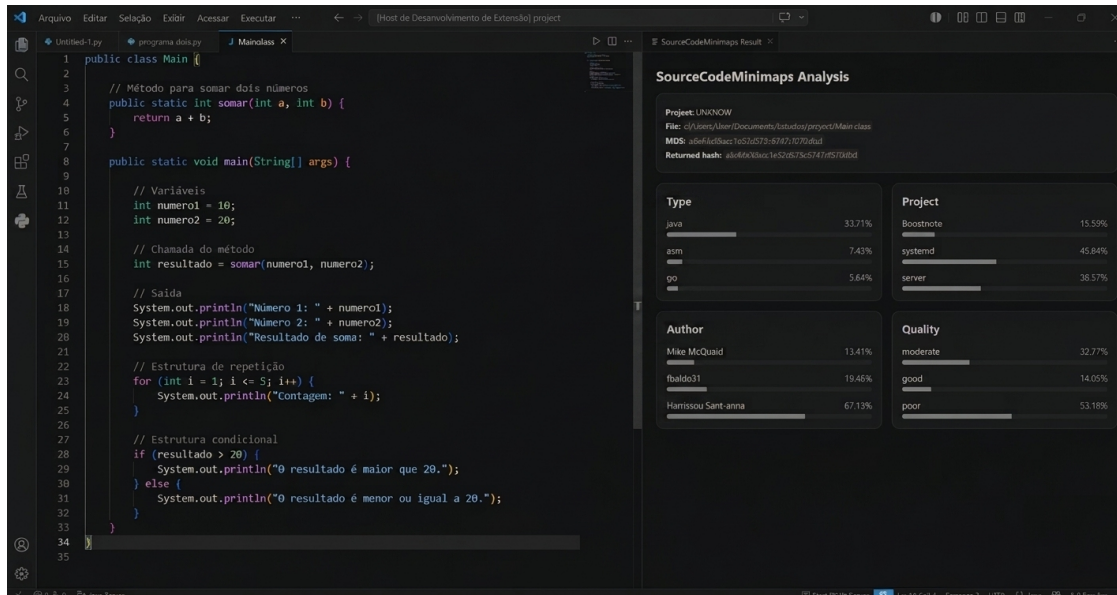
É a evidência de que o sinal discriminativo do minimapa é estrutural, e não lexical.

MinimapSignatures: a abordagem dentro da IDE

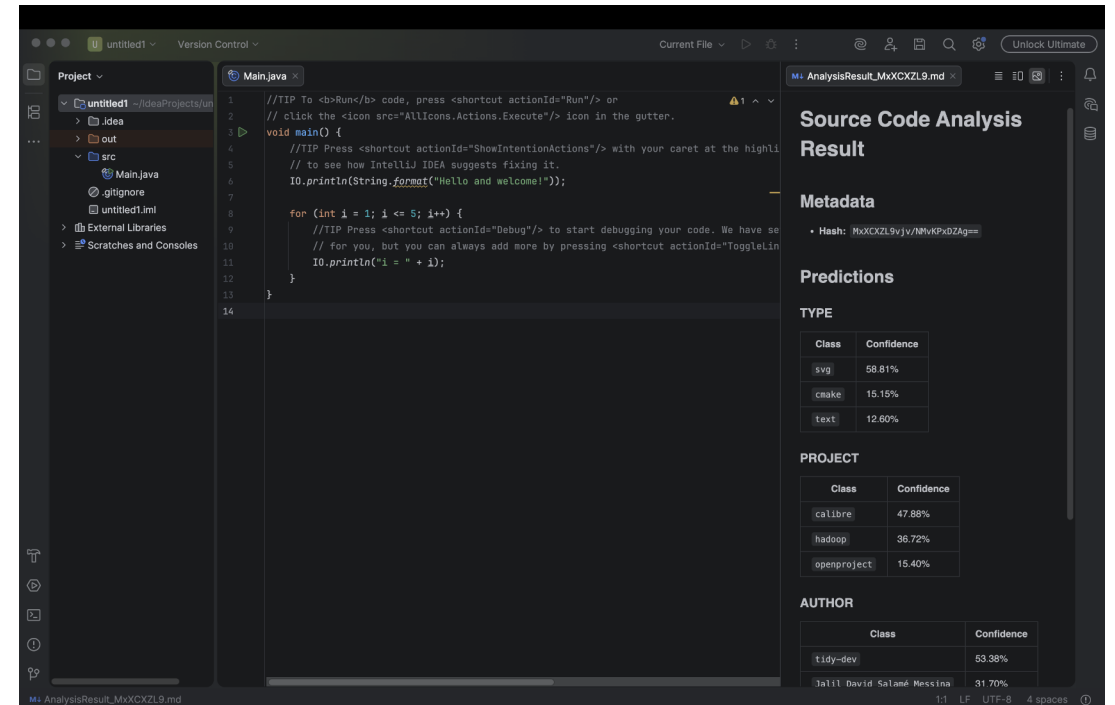
- Backend em FastAPI e PyTorch; clientes para Visual Studio Code e IntelliJ IDEA
- O minimapa é gerado no cliente: só a imagem anonimizada trafega, nunca o código-fonte
- MultiTaskMinimapCNN: tronco convolucional compartilhado, embedding de 256 dimensões e três cabeças (Tipo, Projeto, Autor)
- Contrato compartilhado entre clientes: novos IDEs bastam gerar o mesmo formato
- O alvo de qualidade existe no contrato como placeholder e devolve dado simulado — nenhum modelo foi treinado para ele



Validação funcional de ponta a ponta



Visual Studio Code: WebView com minimapa e previsões



IntelliJ IDEA: relatório em Markdown ao lado do arquivo

O que está validado: captura, geração local, anonimização, comunicação, inferência e retorno. O que não está: utilidade, confiança e efeito no trabalho do desenvolvedor — isso exige estudo com usuários.

Produção científica

Trabalho	Veículo	Situação
Software Feature Detection Using Source Code Minimaps as Visual Signatures	IWSSIP 2025 (IEEE)	Publicado
Source Code Minimaps at Scale: Feature-Oriented Analysis of 100 Projects	CIARP 2025 (Springer, LNCS 16528)	Publicado
MinimapSignatures: A Multi-IDE Ecosystem for Analysis of Visual Source Code Signatures	SBES-Tools 2026 (CBSoft)	Aceito
Explicabilidade de representações visuais de código	VISAPP 2027	Planejado
Estudo consolidado de análise visual com preservação de confidencialidade	Empirical Software Engineering	Planejado

A ablação pareada do Estudo 3 é resultado inédito, produzido para este documento e ainda não submetido.

Limitações que reconheço

- Estudo 1: corpus limitado, partição única e sem estimativa de variância; a métrica reportada é apenas acurácia
- Estudo 2: desbalanceamento severo; partição não disjunta por repositório; macro-F1 e resultados de teste ainda não consolidados
- Estudo 3: um sistema, um descritor, um classificador e um único objetivo — LBP pode ser cego justamente ao que a anonimização remove
- O objetivo Autor é sonda metodológica, não ferramenta de atribuição: serve para mostrar que o estilo sobrevive à supressão lexical
- A ferramenta está validada funcionalmente, não avaliada com desenvolvedores
- Os estudos tratam de classificação; ranqueamento, detecção de anomalia e agrupamento seguem inexplorados

Plano de pesquisa: seis atividades

- A1** **Curadoria do dataset** — higiene de rótulos, arquivos gerados, separação de bots e metadados de repositório
- A2** **Avaliação da anonimização** — replicar em corpus aberto, com redes profundas e nos três objetivos; cinco níveis de supressão
- A3** **Modelagem sensível a desbalanceamento** — pesos, amostragem balanceada, focal loss, ajuste de logits e calibração
- A4** **Multiescala e por região** — resoluções, patches, pirâmides e transformers; identificar trechos e não só o arquivo
- A5** **Qualidade e desvio arquitetural** — code smells, papel arquitetural, arquivos gerados e desvio de convenção
- A6** **Avaliação da ferramenta** — tarefas controladas, entrevistas e observação; análise multiarquivo e sobreposição de explicações

Cronograma até a defesa

Jul – ago 2026	Replicar a ablação A2 no corpus aberto; refinar o catálogo (A1)
Set 2026	Qualificação em 3 de setembro; incorporar o retorno da banca; submeter ao VISAPP; organizar o estágio
Out 2026 – fev 2027	Estágio PDSE na Universidad Rey Juan Carlos (Madri): A1, A3, A4 e protocolos de explicação; VISAPP em fevereiro
Mar – mai 2027	Avaliação disjunta por repositório; consolidar teste e métricas por classe; iniciar a escrita
Jun – ago 2027	Piloto de qualidade (A5); estender a ferramenta; submeter à Empirical Software Engineering
Set – out 2027	Avaliação com desenvolvedores (A6); seguir escrevendo
Nov – dez 2027	Consolidar resultados, fechar o texto, preparar artefatos e a defesa

Contribuições esperadas

- Metodologia reprodutível para análise visual de código com preservação de confidencialidade
- Estudo de anonimização documentado, com curvas de compromisso entre supressão e desempenho
- Avaliação sob partições disjuntas por projeto
- Comparação sistemática entre descritores artesanais e representações aprendidas
- Protocolos de explicabilidade para representações visuais de código
- MinimapSignatures estendido, com análise multiarquivo e avaliação centrada no desenvolvedor

A tese sustenta que, para uma família de tarefas de análise de software, a forma da página basta — e o léxico pode ficar em casa.

Obrigado. Perguntas?